Design Methodology

From Research to Realization — How We Design and Build Prologos

The Prologos Project

2026-02-24

Contents

1 Purpose **2**

2 The Five Phases **2**

2.1 Phase 1: Deep Research . . . 2
2.1.1 What this looks like . . . 3
2.1.2 Key practices . . . 3
2.1.3 Artifacts . . . 3
2.1.4 Example . . . 4
2.2 Phase 2: Research Refinement and Gap Analysis . . . 4
2.2.1 What this looks like . . . 4
2.2.2 Key practices . . . 4
2.2.3 Artifacts . . . 5
2.3 Phase 3: Design Iteration . . . 5
2.3.1 What this looks like . . . 5
2.3.2 Key practices . . . 6
2.3.3 The critique cycle in practice . . . 7
2.3.4 Artifacts . . . 7
2.4 Phase 4: Implementation . . . 8
2.4.1 What this looks like . . . 8
2.4.2 Key practices . . . 8
2.4.3 Implementation flow . . . 9
2.4.4 Artifacts . . . 10
2.5 Phase 5: Composition and Extension . . . 10
2.5.1 What this looks like . . . 11
2.5.2 Key practices . . . 11

3 Cross-Cutting Principles **11**
3.1 Theoretical Grounding, Practical Surface 12
3.2 Design Documents Are Living 12
3.3 Critique Is a Gift 12
3.4 Standups as Design Memory 12
3.5 The 14-File Pipeline as Discipline 12

4 Anti-Patterns **13**
4.1 "We'll come back to it" 13
4.2 Quick fixes that meet criteria partially 13
4.3 Research without documentation 13
4.4 Design without critique 13
4.5 Implementation without tests 13

5 Relationship to Other Principles Documents **14**

6 Exemplar Projects **14**
6.1 Extended Spec Language Design 14
6.2 Logic Engine on Propagators 14
6.3 Collections Ergonomics 15

1 Purpose

This document describes the methodology we follow when designing and implementing significant features in Prologos. It is both a process guide and a set of meta-lessons about *how to do design work well* in a language with deep theoretical foundations and practical engineering constraints.

The methodology emerged organically from our practice on major efforts including the Extended Spec Language Design, the Logic Engine on Propagators, Collections Ergonomics, the Numerics Tower, and the Homoiconicity phases. It is descriptive (capturing what actually works) as much as prescriptive (defining what we should do).

2 The Five Phases

2.1 Phase 1: Deep Research

Goal: Ground ourselves in the best techniques — cutting-edge and well-proven — with a comprehensive survey of the landscape.

2.1.1 What this looks like

- Read primary literature (papers, specifications, reference implementations)
- Survey existing systems that address the same problem space
- Identify the theoretical foundations that our implementation will rest on
- Produce a research document (`docs/research/` or `docs/tracking/`) that synthesizes findings

2.1.2 Key practices

- **Cast a wide net**: Survey at least 3–5 existing approaches to the same problem. For the Extended Spec Design, we surveyed Clojure Spec, Malli, PropEr, Agda, Idris 2, Lean 4, Haskell, F*, Koka, and Racket Contracts before writing a line of design prose.
- **Identify the essential vs. accidental**: What is fundamental to the problem domain vs. what is incidental to a particular implementation? The Logic Engine research distilled that propagator networks, lattice theory, and truth maintenance are the essential components — the specific data structures (CHAMP vs. HAMTs, persistent vs. mutable) are implementation choices.
- **Name the theories**: Lattice theory, Curry-Howard correspondence, the pi-calculus, Radul-Sussman propagators, Kuper's LVars — naming the theoretical foundations makes them citable, shareable, and critiquable. A design grounded in named theory is more robust than one grounded in intuition alone.
- **Document as you go**: Research that isn't written down evaporates when context windows reset. The research document becomes the institutional memory.

2.1.3 Artifacts

- Research document: `docs/research/TOPIC.md` or `docs/tracking/YYYY-MM-DD_TOPIC_RESEARCH.`
- Bibliography of key papers and systems surveyed
- A vocabulary of concepts that the team will use in subsequent phases

2.1.4 Example

The Logic Engine began with `2026-02-23_LATTICE_PROPAGATOR_RESEARCH.md`, covering lattice foundations, LVars/LVish deterministic parallelism, the "Multiverse Mechanism" and "Pocket Universe" theories, Radul-Sussman propagator networks, and TMS/ATMS truth maintenance. This document established the conceptual vocabulary (cells, propagators, quiescence, monotonic merge) that all subsequent design work referenced.

2.2 Phase 2: Research Refinement and Gap Analysis

Goal: Refine our understanding, identify gaps in current infrastructure, consider tradeoffs, identify opportunities aligned with core principles, and make qualified recommendations.

2.2.1 What this looks like

- Review research findings against our existing implementation
- Identify what infrastructure we already have and what is missing
- Enumerate alternative approaches with their tradeoffs
- Check alignment with principles in `DESIGN_PRINCIPLES.org` and `LANGUAGE_VISION.org`
- Make recommendations, with rationale, for which approach to pursue

2.2.2 Key practices

- **Infrastructure gap analysis**: Before designing the Logic Engine, we identified that we needed persistent data structures (CHAMP), lattice traits, and type-system exposure of runtime values — and that we already had CHAMP maps, the trait system, and the 14-file AST pipeline. The gap analysis shapes the roadmap.
- **Tradeoff matrices**: When multiple approaches exist, make the tradeoffs explicit. Mutable vs. persistent propagator networks? The tradeoff is performance (mutable wins for raw speed) vs. backtracking simplicity (persistent wins — O(1) backtrack is "keep old reference"). Making this explicit lets us choose with eyes open.
- **Principle alignment check**: Every recommendation should pass the "does this uphold our principles?" test:

  - Does it maintain homoiconicity?
  - Does it support progressive disclosure?
  - Does it decomplect orthogonal concerns?
  - Does it compose with the existing layered architecture?
  - Does it follow the "most generalizable interface" principle?

  The Logic Engine's decision to subsume LVars into propagator cells (rather than having separate LVar types) was driven by decomplection: cells with per-cell merge functions are the more general abstraction.

- **Opportunities over features**: Don't just fill gaps — identify opportunities where our unique combination of features enables something no other system offers. The fusion of propagators with session types for distributed constraint solving is an opportunity that arises from having both systems in the same language.

2.2.3 Artifacts

- Gap analysis section in the design document
- Tradeoff tables with explicit criteria
- Recommendation with rationale tied to principles

2.3 Phase 3: Design Iteration

Goal: Produce a concrete design with a detailed, phased roadmap for implementation, refined through critique and feedback until there is full clarity.

2.3.1 What this looks like

This is an iterative cycle:

1. **Draft**: Produce a complete design document with implementation roadmap
2. **Critique**: Subject the design to rigorous, independent critique
3. **Respond**: Address critiques — accept, refine, or justify
4. **Repeat**: Continue until all parties have clarity and confidence

2.3.2 Key practices

- **Comprehensive first drafts**: The first draft should be as complete as possible. Include data structures, type signatures, AST nodes, phase dependencies, test strategies, and concrete code examples. An incomplete draft invites incomplete critique.

- **Invite adversarial critique**: Seek critique that probes weaknesses, not confirmation that validates strengths. The Logic Engine design was subjected to a 10-point critique covering resolution strategy (underspecified), contradiction detection (hand-wavy), negation handling (unsound without stratification), and performance analysis (missing). Every point led to a design improvement.

- **Distinguish design issues from implementation issues**: A critique that says "the resolution strategy is unclear" is a design issue that must be resolved before implementation. A critique that says "CHAMP lookup is O(log n)" is an implementation detail that can be optimized later. Don't let implementation concerns block design progress, but don't let design ambiguity persist into implementation.

- **Pushback with context**: Not all critiques are valid. When a critique misunderstands the system's context, push back with the fuller picture. When the Logic Engine critique suggested adding `:includes` to traits for contradiction detection, we pushed back: "We don't want trait hierarchies AT ALL, EVER! This is the whole rationale for the `bundle` concept." The pushback refined both the design and our articulation of the principle.

- **Phase dependencies are architecture**: The phased roadmap isn't just a schedule — it's an architectural statement about what depends on what. The Logic Engine's critical path (`Phase 1 → 2 → 3 → 5 → 6 → 7`, with `Phase 4` parallelizable) encodes fundamental architectural dependencies. Getting this right prevents wasted work.

- **Concrete over abstract**: Show elaboration examples, not just type signatures. Show how `defr ancestor` becomes propagator cells and fire functions, not just "relations elaborate to propagator networks." Every level of abstraction should have at least one concrete example that a reader can trace through.

2.3.3 The critique cycle in practice

The Extended Spec Design went through multiple rounds:

1. **Initial research survey** → comprehensive landscape analysis
2. **First design draft** → keyword symmetry table, metadata map design, `property` as composable proposition groups
3. **Feedback round** → Should `property` use `:includes` or `:extends`? How do properties attach to traits? Resolution: `:includes` for conjunctive composition (set union), `:laws` on trait for attachment.
4. **Final refinement** → phased roadmap separating syntax (Phase 1) from backends (Phase 2+), acknowledging what can be built now vs. what needs future infrastructure.

The Logic Engine Design went through a similar cycle:

1. **Research** → lattice/propagator theory survey
2. **Vision discussion** → relational language syntax, solver architecture, provenance, tabling-by-default
3. **First design draft** → 7-phase roadmap, data structures, AST nodes
4. **Independent critique** → 10 points of feedback, including fundamental gaps in resolution strategy and functional-relational boundary
5. **Response and refinement** → accepted `HasTop` trait (Point 3), rejected trait hierarchy (Point 3 alternative), clarified UnionFind vs. cells, added resolution strategy section, added performance expectations
6. **Final roadmap** → phased implementation plan with clear phase dependencies and test strategies per phase

2.3.4 Artifacts

- Design document: `docs/tracking/YYYY-MM-DD_TOPIC_DESIGN.org` (org-mode preferred for structured prose with code blocks)
- Phased implementation roadmap with explicit dependencies
- Phase-specific test strategies and success criteria
- Record of critique and responses (preserved in standups)

2.4 Phase 4: Implementation

Goal: Execute the phased roadmap, shipping complete and sound solutions at each phase, raising design issues as they surface.

2.4.1 What this looks like

- Work through the roadmap phase by phase
- Create tracking documents before implementation begins
- Ship complete, tested solutions — not partial scaffolding
- When implementation reveals design gaps, stop and address them

2.4.2 Key practices

- **Completeness over deferral**: This is our most important implementation principle (see `DEVELOPMENT_LESSONS.org` § "Completeness Over Deferral"). When you have clarity, vision, and full context — finish the work now. Half-built pieces that get deferred are half-built pieces that get forgotten. The cost of re-acquiring context later almost always exceeds the cost of doing the work while the understanding is fresh.

  Defer *only* when there is a genuine dependency on unbuilt infrastructure or genuinely uncertain design. "We'll come back to it" is a red flag.

- **Design issues surface during implementation**: This is expected and healthy. The Logic Engine Phase 3 revealed that:

  - Parametric `impl` dispatch failed for compound type args without `where` constraints (a `macros.rkt` fix was needed)
  - Generic trait accessor calls inside closures can't resolve implicit type params (a genuine meta-resolution limitation, deferred with explicit tracking)
  - QTT multiplicity violations arise from inline lambdas that reference prelude functions differently from named definitions

  Each of these was addressed: the first two with code fixes and design documentation, the third with a workaround and a tracked limitation. The key is: *don't paper over design issues with quick fixes that meet criteria partially.*

- **If a gap requires deeper infrastructure, address the core concern**: When the Logic Engine critique identified that contradiction detection needed a `HasTop` trait, we built it — including a `BoundedLattice` bundle and trait instances for Bool, FlatVal, and Interval. This was more work than a simple boolean flag, but it produced the correct abstraction that composes with the rest of the trait system.

- **Phase-gated implementation with sub-phases**: Break phases into lettered sub-phases (a, b, c. . .) with explicit "done" vs. "remaining" tracking. The Logic Engine Phase 3 was broken into:

  - 3a: AST structs + mechanical traversals
  - 3b: Type rules + type tests
  - 3c: Reduction rules + eval tests
  - 3d: Surface syntax + integration tests
  - 3e: Library wrappers + trait tests

  Each sub-phase has a clear scope, produces testable artifacts, and creates a natural commit point.

- **Test at every boundary**: Every sub-phase should end with passing tests. The Logic Engine implementation produced 56 new tests across 3 test files, verifying types (32 tests), integration (16 tests), and library-level traits (8 tests). Tests are not an afterthought — they are the proof that the implementation matches the design.

- **Track deferred work immediately**: When something must be deferred (the `new-lattice-cell` generic wrapper, for instance), add it to `DEFERRED.md` in the same commit. Deferred work not tracked is abandoned work.

2.4.3 Implementation flow

```
Create tracking document
(before writing any code)



Sub-phase a: Foundation          Tests pass?  Yes  Commit
(structs, types, basic cases)
```

```
        No


Sub-phase b: Core logic              Fix / Investigate
(type rules, reduction, etc.)




Design issue discovered?          Stop. Address it.
                                  Yes  Update design doc
     Then continue.
                No


Sub-phases c, d, e...
(repeat until phase complete)



Full test suite + whale check
Update DEFERRED.md
Update tracking document
Commit
```

2.4.4 Artifacts

- Phase-specific commits with descriptive messages
- Updated tracking document with completion status
- Updated `DEFERRED.md` for any deferred work
- Test suite growth (measurable)

2.5 Phase 5: Composition and Extension

Goal: Enjoy the well-thought-out design and how it composes with our multi-level, modular, extensible language. Verify that the new feature integrates cleanly and enables future extension.

2.5.1 What this looks like

- Verify that the new feature composes with existing features
- Check that the new abstractions are reusable in contexts beyond the original design
- Update principles documents if new patterns emerged
- Identify the new possibilities that the feature enables

2.5.2 Key practices

- **Composition is the test**: A well-designed feature should compose with features it wasn't specifically designed for. The propagator network composes with the trait system (lattice traits define merge behavior), with the type system (PropNetwork is a first-class type), and with the prelude (users can write lattice-aware programs with standard library functions). If a feature *doesn't* compose, that's a design smell worth investigating.
- **Extension paths are visible**: After shipping Phase 3 of the Logic Engine, the path to Phase 4 (UnionFind), Phase 5 (ATMS), and Phase 7 (surface syntax) is clear. Each depends on the infrastructure just built, and the design document already specifies what each will need. A good implementation *opens doors* rather than closing them.
- **Update the living documents**: When a feature completes, update:
  - `DEFERRED.md` (mark phases complete, add new deferrals)
  - `MEMORY.md` (project memory for context recovery)
  - Relevant principles docs (new patterns, new lessons)
  - The original tracking document (completion status, lessons learned)
- **Teach through usage**: Write examples that show the feature in context. The Logic Engine's test files serve double duty as documentation: they show how to create networks, add cells, wire propagators, run to quiescence, and read results. Future users (including future context windows) learn from these examples.

3 Cross-Cutting Principles

These principles apply across all five phases.

3.1 Theoretical Grounding, Practical Surface

Every design decision should be traceable to a theoretical foundation, but the surface presentation should be practical and approachable. "Lattice-theoretic monotonic constraint propagation" is the theory; `net-cell-write` with a merge function is the practice. Both must exist: the theory ensures correctness, the practice ensures usability.

3.2 Design Documents Are Living

Design documents are not write-once artifacts. They are updated as implementation reveals new information. The Logic Engine design document was refined after every critique round and after Phase 3 implementation revealed the `HasTop` trait need and the implicit-resolution limitation. Stale design documents are worse than no design documents.

3.3 Critique Is a Gift

Adversarial critique — even harsh critique — improves designs. The 10-point critique of the Logic Engine identified real gaps (resolution strategy underspecified, negation handling unsound). Every accepted critique made the design stronger. The discipline is to engage with critique on its merits, not defensively.

Not all critiques are valid. The key judgment is: does this critique identify a real gap in the design, or does it misunderstand the system's context? When the latter, push back clearly and use the pushback to sharpen your articulation of *why* the design is the way it is.

3.4 Standups as Design Memory

Our standup documents (`docs/standups/standup-YYYY-MM-DD.org`) serve as a chronological record of design discussions, including the back-and-forth of critique and refinement. The (human) and (machine) annotations preserve the conversational flow. This record is invaluable for reconstructing design rationale months later.

3.5 The 14-File Pipeline as Discipline

Prologos's 14-file AST pipeline (syntax → surface-syntax → parser → elaborator → typing-core → qtt → reduction → substitution → zonk → pretty-print → unify → macros → foreign) is both a cost and a benefit. The cost is

that every new AST node requires touching many files. The benefit is that every subsystem handles every node consistently. This enforced consistency is a design methodology in itself: if a feature can't be expressed through the pipeline, it's a signal that the feature's abstraction is wrong.

4 Anti-Patterns

4.1 "We'll come back to it"

Defer only when genuinely blocked. Track all deferrals immediately. See `DEVELOPMENT_LESSONS.org` § "Completeness Over Deferral."

4.2 Quick fixes that meet criteria partially

A partial solution that passes tests but doesn't address the underlying design concern creates technical debt with interest. Better to identify the design gap, address it, and ship the complete solution.

4.3 Research without documentation

Research that exists only in a developer's head is lost when context resets. Write it down. Even rough notes are better than nothing.

4.4 Design without critique

A design that hasn't been subjected to adversarial critique has unknown weaknesses. Actively seek feedback, especially on the parts you're most confident about — confidence is where blind spots hide.

4.5 Implementation without tests

An untested implementation is an unverified hypothesis. Tests are the proof that the implementation matches the design. New features aren't done until they have tests.

5 Relationship to Other Principles Documents

Document	Relationship to this methodology
`DESIGN_PRINCIPLES.org`	Provides the values that Phase 2 alignment checks use
`LANGUAGE_VISION.org`	Provides the north star that research (Phase 1) aims at
`DEVELOPMENT_LESSONS.org`	Collects lessons that emerge from Phase 4
`PATTERNS_AND_CONVENTIONS.org`	Captures patterns that Phase 5 discovers
`ERGONOMICS.org`	Informs Phase 2 tradeoff analysis for user-facing features
`RELATIONAL_LANGUAGE_VISION.org`	Example of Phase 1-2 output for a specific domain

6 Exemplar Projects

These completed efforts illustrate the methodology in practice:

6.1 Extended Spec Language Design

- **Research**: Survey of 10+ systems (Clojure Spec, Malli, PropEr, Agda, Idris 2, Lean 4, Haskell, F*, Koka, Racket Contracts)
- **Design**: `2026-02-22_EXTENDED_SPEC_DESIGN.org` with keyword symmetry table, metadata map design, 4-phase roadmap
- **Iteration**: Multiple rounds on `property` composition, `:includes` semantics, functor metadata
- **Implementation**: Phase 1 complete (`??` holes, `property`, `functor`, `:examples`, `:deprecated`). Phases 2+ tracked in DEFERRED.md.
- **Composition**: `property` composes with `trait` via `:laws`, `bundle` composes `property` groups, `functor` composes with `spec` metadata.

6.2 Logic Engine on Propagators

- **Research**: `2026-02-23_LATTICE_PROPAGATOR_RESEARCH.md` + the broader `TOWARDS_A_GENERAL_LOGIC_ENGINE_ON_PROPAGATORS.org` synthesis
- **Design**: `2026-02-24_LOGIC_ENGINE_DESIGN.org` with 7-phase roadmap, 14 AST nodes, data structure specifications
- **Iteration**: Relational language vision discussion, design critique (10 points), pushback on trait hierarchies, refinement of resolution strategy and HasTop design

- **Implementation**: Phases 1–3 complete. Phase 1 (lattice traits), Phase 2 (persistent PropNetwork with BSP), Phase 3 (14 AST nodes through 12-file pipeline, HasTop trait, BoundedLattice bundle). 56 new tests, parametric impl dispatch fix.
- **Composition**: PropNetwork composes with trait system (Lattice, HasTop, BoundedLattice), type system (first-class type), prelude (available from any `ns` module).

6.3 Collections Ergonomics

- **Research**: Audit of existing collection generics, gap identification
- **Design**: Stages A–I with clear boundaries
- **Implementation**: Stages A–H complete in one session (completeness over deferral). Stage I deferred (genuine infrastructure dependency on transient type exposure).
- **Composition**: Generic `map~`/`~filter~`/`~reduce` work across all collection types. `into` converts between any two collections.